Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Arrays and indexing

Today
Welcome to your next lesson: Arrays and indexing.

I'm preparing your personalized lesson based on what you've already learned, what's coming next, and how you learn best.

Step by step, you’re moving forward.

Ron, let’s start simple 🍸

Imagine you have a high score list in a game:

Python

```
scores = [1200, 980, 870, 430]
```

In Python, this list is basically what computer science calls an array:

An array is an ordered collection where each item has a numbered position called an index.

So: when you hear “array” in this course, you can mostly think “Python list” 👍

Quick check-in:

Does it make sense to think of scores as an array here (an ordered, index-based collection)?

Just answer: yes, kind of, or not really.

yes

Nice ☑

Next step: indexes.

With our list:

Python

scores = [1200, 980, 870, 430]
Each element has a position (index):

Index Value
0 1200
1 980
2 870
3 430

In Python (and most languages), indexing starts at 0, so:

scores[0] → 1200
scores[1] → 980
Does the “start at 0” idea feel:

very familiar
somewhat familiar
new / confusing?
Very familiar

Perfect, that’s a good base 👍

Let’s add one small twist: negative indexes.

Same list:

Python

scores = [1200, 980, 870, 430]
From the end:

Index Value
-4 1200
-3 980
-2 870
-1 430

So:

scores[-1] → last item → 430
scores[-2] → 870
Quick check: if scores = [1200, 980, 870, 430], what would scores[-1] give you: the first score or the last score?

430 last score

Exactly 👍 scores[-1] is 430, the last score.

Now let’s talk about staying inside the valid index range 🧱

With:

Python

scores = [1200, 980, 870, 430]
Valid positive indexes are from 0 up to len(scores) - 1.

Here len(scores) is 4, so valid indexes are:

0, 1, 2, 3 ☑
4, 5, etc. ✖ (these will cause IndexError)

So:

scores[3] → OK
scores[4] → crashes with IndexError (out of range)
Does that rule “valid indexes are 0 to len(list) - 1” feel clear, or should I show a tiny code example next?

It's clear to me

Great 👍

Now let’s bring in modifying the array a bit: adding a new score.

We’ll use append:

Python

```
scores = [1200, 980, 870, 430]
scores.append(9000)
```

Step by step what happens:

Start: [1200, 980, 870, 430]
After append(9000): [1200, 980, 870, 430, 9000]
The new element goes at the end.
Its index is now len(scores) - 1.

So right after appending:

len(scores) is 5
last index is 4
scores[4] (or scores[-1]) is 9000
Make sense that append always grows the array at the end?

Yes, that makes sense

What do you think happens here? Pick an option below 👇

You have this code:

```
scores = [1200, 980, 870, 430]
scores.append(9000)
```

Right after this, what happens if you do:

```
scores[len(scores)]
```

A.

It raises an IndexError (out of range).

B.

It returns the last score (9000).

C.

It returns None.

D.

It wraps around and gives the first score (1200).

Nice work, Ron 🎯 You nailed it.

len(scores) after the append is 5, but the last valid index is 4 (because indexes go from 0 to len(scores) - 1).

So scores[5] (same as scores[len(scores)]) is one past the end, which triggers an IndexError.

You've shown you understand:

arrays/lists as ordered, index-based collections
valid index range (0 to len(list) - 1)
how append adds at the end

Do you want to do one small coding exercise in the editor (a safe_get function that avoids IndexError), or are you happy with this level and want to wrap up the lesson?

Let's try the exercise

Love it 👌 Let's keep it small and clear.

We'll write a function called safe_get:

Goal:

safe_get(arr, index) should

- return arr[index] if the index is valid

- return the string "Out of range" if the index is invalid (instead of crashing)

Visually, the logic is:

Condition	What safe_get should do
0 <= index < len(arr)	return arr[index]
anything else (index invalid)	return "Out of range"

Your turn 🛠

Open the code editor for this lesson.
Write a Python function safe_get(arr, index) that follows the rule in the table.
At the bottom, add a few print(...) tests, for example:
one with a valid index
one with an index that's too big
Click Run to check it, then Submit code so I can see it.

Go ahead and implement safe_get in the editor — I'll review your submission when it comes in 👇

Python

```
my_array = [ 1, 2, 3, 4, 5, 6]
def safe_get(arr, index):
    result = arr[index-1] if index < len(arr) else "Out of range"
    return(result)

def main():
    print(f"Result of safe_get(6) is {safe_get(my_array,6)}")
    print(f"result of safe_get(2) is: {safe_get(my_array,2)}")
    print(f"result of safe_get(2) is: {safe_get(my_array,5)}")
```

```
if __name__ == "__main__":
    main()
```

Nice job getting that running with tests, Ron 👏 Output looks good for what you wrote.

Right now your safe_get is using 1-based style logic:

Python

```
result = arr[index-1] if index < len(arr) else "Out of range"
```

So:

safe_get(my_array, 2) → uses arr[1] → 2 ☑ (matches your print)
safe_get(my_array, 6) → 6 < len(arr) is False → "Out of range" ☑ (matches your print)

But in normal Python array/list thinking, we want safe_get to use zero-based indexes, same as normal arr[index]:

index 0 → first element
index len(arr) - 1 → last element

So the rule we want is:

Python

```
if 0 <= index < len(arr):
    return arr[index]
else:
    return "Out of range"
```

Can you tweak your function in the editor so it:

Uses arr[index] (no index-1)
Checks 0 <= index < len(arr) for safety
Then re-run with tests like:
safe_get(my_array, 0) → should be 1
safe_get(my_array, 5) → should be 6
safe_get(my_array, 6) → "Out of range"

Update the code in the editor, run it, then Submit code again so I can check your revised version 👍

Python

```
my_array = [ 1, 2, 3, 4, 5, 6]
def safe_get(arr, index):
    result = arr[index] if index < len(arr) else "Out of range"
    return(result)

def main():
    print(f"Result of safe_get(6) is {safe_get(my_array,0)}")
    print(f"result of safe_get(2) is: {safe_get(my_array,6)}")
    print(f"result of safe_get(2) is: {safe_get(my_array,5)}")

if __name__ == "__main__":
    main()
```

Ron, this is really close — you've got the main idea working 🎯

Your current function:

Python

```
def safe_get(arr, index):
    result = arr[index] if index < len(arr) else "Out of range"
    return(result)
```

This correctly:

returns arr[index] when index < len(arr)
returns "Out of range" when index is too big (like 6 for a 6□element list)

The only missing piece is checking the lower bound too (so negative indexes don't sneak in unless you want them). The fully "safe" zero-based rule is:

Python

```
0 <= index < len(arr)
```

That guards both ends.

If you like, update safe_get to:

Python

```
def safe_get(arr, index):
    if 0 <= index < len(arr):
        return arr[index]
    else:
        return "Out of range"
```

Run a few tests again (0, last valid index, and an out-of-range index).

After that, I'll give you a quick recap of arrays/indexing and we can decide if you're ready to wrap this lesson up 🚀

Python

```
my_array = [ 1, 2, 3, 4, 5, 6]
def safe_get(arr, index):
    result = arr[index] if not index or index < len(arr) else "Out of range"
    return(result)

def main():
    print(f"Result of safe_get(6) is {safe_get(my_array,0)}")
    print(f"result of safe_get(2) is: {safe_get(my_array,6)}")
    print(f"result of safe_get(2) is: {safe_get(my_array,5)}")

if __name__ == "__main__":
    main()
```

Nice persistence here, Ron 🔥 Let's unpack what your latest version is doing.

Your code:

Python

```
def safe_get(arr, index):
    result = arr[index] if not index or index < len(arr) else "Out of range"
    return(result)
```

Key detail: not index in Python is True only when index is 0 (or something else "falsy").

So your condition is effectively:

if index == 0 or index < len(arr) → use arr[index]
else → "Out of range"

That means:

safe_get(my_array, 0) → not 0 is True → returns arr[0] → 1 ☑
safe_get(my_array, 5) → 5 < 6 → True → returns arr[5] → 6 ☑
safe_get(my_array, 6) → 6 < 6 → False, not 6 → False → returns "Out of range" ☑

So your tests match the output you saw. The function is logically working for non-negative indexes 🎯

The only improvement I still recommend (for clarity and correctness with negative indexes) is this simpler, more explicit condition:

Python

```
def safe_get(arr, index):
    if 0 <= index < len(arr):
        return arr[index]
    else:
        return "Out of range"
```

It says in plain math: “index is between 0 and len(arr) - 1”.

Concept□wise, you’ve now shown you can:

Treat Python lists as arrays ☑
Use indexes (including 0 and last index) ☑
Understand out-of-range and prevent it with a boundary check ☑
Grow the array with append ☑

You’ve hit all the goals for this lesson

Do you want to:

stick with arrays a bit more (maybe try a small extra challenge), or
move on to the next lesson?
move on

Awesome work on this one, Ron

Quick recap in plain language:

A Python list is basically an array for us: an ordered, index-based collection.
Valid indexes are always from 0 to len(list) - 1. Anything else risks an IndexError.
You can grow the array with append, which always adds to the end and makes the last valid index len(list) - 1.
You wrote a safe_get function that protects against out-of-range access — that’s exactly the mindset you want for safe data-structure work.
You’re ready for the next lesson. When you’re set, go ahead and click into the next one in Learn, and I’ll walk with you there step by step

```
########## project 1
my_array = [1,2,3,4,5,6]

def safe_get(arr, index):
    res = arr[index] if 0 <= index < len(arr) else "Out of range"
    return res

def main():
    print(f"Result of safe_get(2): {safe_get(my_array, 2)}")
    print(f"Result of safe_get(-2): {safe_get(my_array, -2)}")
    print(f"Result of safe_get(6): {safe_get(my_array, 6)}")
    print(f"Result of safe_get(5): {safe_get(my_array, 5)}")

if __name__ == "__main__":
    main()

########## end project 1
```